GPU-Accelerated FM Radio Demodulator

Real-time IQ to Audio pipeline on the NVIDIA Jetson Orin Nano

Gage Shaddock

CS147 GPU Programming

High-Level Project Idea

RTL-SDR -> CUDA Pipeline -> Demodulated Audio

What it does

- Captures raw IQ samples from an RTL-SDR V4 USB dongle at 2.048 Msps
- Processes each 131,072-sample block before the next one arrives (~64 ms window)
- Demodulates FM radio to 48 kHz mono audio in real time
- Multi-channel version decodes 4 FM stations simultaneously

Hardware

- NVIDIA Jetson Orin Nano with unified CPU+GPU memory
- RTL-SDR V4 USB dongle provides IQ sample source
- Shared buffers allows for zero-copy transfers

Key design insight

The Jetson's unified memory lets the RTL-SDR callback write directly into a buffer the GPU can read, no cudaMemcpy needed. All six pipeline stages run in a single CUDA stream.

Six-Stage CUDA Kernel Pipeline

Each stage parallelized across the GPU

1

Normalize

uint8 → float [-1,1]
1 thread / IQ pair

2

Freq Shift

Shift target → 0 Hz
1 thread / IQ pair

3

LPF + Decimate

100 kHz cutoff
÷7 decimation (keep every 7th)

4

FM Demod

Phase diff per sample
1 thread / decimated sample

5

DC Removal

Parallel reduction sum then subtract from signal

6

LPF + Decimate

Audio bandwidth filter
Final 48 kHz output

Implementation

Four versions built, benchmarked, and compared

CPU

Sequential Reference

Pure C, single-threaded. Runs on the Jetson for a fair baseline. Achieves 8.4× real-time headroom.

GPU Naive

Initial GPU Port

Separate I and Q kernels, no shared memory, more kernel launches. Validates GPU correctness.

GPU Optimized

Optimized GPU version

Combined I/Q kernels, shared-memory tiling on LPF + FM demod, parallel reduction DC removal.

Multi-Channel

4-Channel Streams

One CUDA stream per station, offsetting center frequency. Decodes 4 FM stations at once in real time.

Results

Block = 131,072 samples · collected every 64 ms · lower is faster

8.4× CPU real-time headroom
7.58 ms · 99.9% CPU

43.9× GPU Naive real-time speedup
1.46 ms · 16% CPU

181× GPU Opt real-time speedup
0.35 ms · 67% CPU

56× Multi-ch speedup (5 stations)
1.14 ms · 57% CPU

Challenges & What I Learned

01 Signal processing from scratch

Had to learn FM math (phase differentiation, decimation, DC removal) before writing a single kernel. Reading the RTL-SDR librtlsdr docs to get the dongle running on the Jetson took dedicated time.

02 GPU warm-up distortion

Initial naive version produced distorted audio. Root cause: the GPU wasn't warmed up. Fixed by running one dummy pipeline pass in the create() function before real data arrives.

03 Shared memory + inter-kernel state

FM demod needs overlapping samples across kernel launches. Solved by adding a persistent state struct that stores boundary values and has the first thread reload them each call.

04 Parallel reduction for DC removal

A single-kernel reduction + subtraction caused race conditions. Split into two kernels: one that accumulates the sum via parallel reduction, and one that normalizes and subtracts.

05 Multi-stream synchronization

Reworked the host/GPU interface for multi-channel: one stream per station, updated struct to hold per-channel state, resolved sync issues between streams sharing the input buffer.

Demo

What the demo shows

- Run demo.sh: builds and executes all four versions against the same 60-second IQ capture
- Live timing table printed at the end: per-block time and real-time speedup for each version
- Single-station playback: demodulated audio output as .wav
- Multi-channel version: five simultaneous station outputs at different kHz offsets

Commands

Build + run all versions

```
./demo.sh
```

Naive — tune to 95.1 MHz

```
./fm_rx_gpu_naive
  -f 95100000 -o out.wav
```

Optimized version

```
./fm_rx_gpu_opt
  -f 95100000 -o out.wav
```

5-channel decode

```
./fm_rx_gpu_multi -f 89100000
-c "-800000,0,600000,1000000"
-o data/out -t 10
```